AZ AI ÚTVONALTERVEZŐ ALGORITMUS

Az algoritmus alapelvei

1. A rover csak olyan célt választ, amelyhez biztonságosan oda tud menni, ott szükség esetén tud bányászni, és utána vissza is tud térni.
2. A rover minden lépésnél figyelembe veszi:
   - az akkumulátort
   - nappal/éjszaka ciklust
   - a hátralévő időt
   - a hazajutás lehetőségét (ez a legfontosabb cél a programomban)
   - a kibányaszható érmeket
3. A rover nemcsak a pillanatnyi legjobb műveletet nézi, hanem előre ellenőrzi, hogy a teljes terv végrehajtható-e.

Ezért a rendszer nem egyszerűen útkereső algoritmus (A* ebben az esetben), hanem egy biztonságos tervellenőrző és döntéshozó logika.

A döntési rendszer

A kódomban a következő funkciókkal találkozhatnak:

1. Hazatérési távolság előszámítása

   hazaTavBFS

2. Útvonal keresése két pont között (A* algoritmus)

   astar

3. Minimális szükséges idő becslése egy teljes útra (utesek = (óra)ütések)

   teljesUtUtes

4. Egy teljes terv szimulációja és megvalósíthatóságának ellenőrzése

   tervSzimulacio

5. Biztonságos célpont kiválasztása

celpontValasztas

6. Biztonságos sebesség kiválasztása az aktuális körben (minimalizáltam a 3. sebesség használatát)

biztSebesseg

7. Az aktuális kör tényleges végrehajtása

szimulacioLepes

Magas szintű AI stratégia

A rover minden körben a következőket próbálja végrehajtani:

4. Ha éppen egy ásványon áll: ellenőrizze, hogy most biztonságos-e bányászni – ha igen, bányásszon.
5. Ha nem bányászik: keressen egy biztonságosan vállalható ásványcélt.
6. Ha van ilyen cél: keressen hozzá útvonalat, és válasszon olyan sebességet, amely után a teljes terv még mindig biztonságos.
7. Ha nincs biztonságos ásványcél, vagy oda most nem lehet biztonságosan elindulni: próbáljon hazafelé menni.
8. Ha a hazafelé mozgás sem lehetséges biztonságosan: várakozzon (nincs elegendő energia a mozgáshoz).

Pszeudokód: teljes AI döntési folyamat

Eljárás: RoverAI_KoronkentiDontes (valós megfelelő: szimulacioLepes)

Bemenet:

rover aktuális állapota

küldetés adatai

térkép

hazatérési távolságok

log fájlok

Kimenet:

kör végrehajtása

rover állapotának frissítése

naplózás (adatok betöltése a fájlba, amit majd a GUIhoz használok fel)

Pszeudokód:

```
számold ki a hazatérési távolságot az aktuális pozícióból (haz_tav)
HA a hazatérési távolság nem ismert (255) AKKOR
  ugorj a HAZAFELÉ blokkra
VÉGE
HA a hátralévő idő <= (hazatérési távolság / max_sebesség) + 1 AKKOR
  ugorj a HAZAFELÉ blokkra
VÉGE
HA a rover ásvány mezőn áll ÉS bányászás megengedett (banyaszhat) AKKOR
  számold ki a bányászás utáni akkut (banyaszFunc)
  ellenőrizd, hogy a bányászás után még biztosan haza lehet-e jutni (tervSzimulacio)
  HA igen AKKOR
    végezd el a bányászatot
    frissítsd az ásványdarabszámot
    töröld az ásványt a térképről
    frissítsd az akkut
    naplózd a DIGGING műveletet
    lépj a következő körre
  VÉGE
VÉGE
keress biztonságos ásványcélt (celpontValasztas)
HA van biztonságos cél AKKOR
  keress pontos útvonalat a célhoz (astar)
  HA van útvonal AKKOR
    számold ki, hány lépés van még a célig
    válassz biztonságos sebességet (biztSebesseg)
    HA létezik megfelelő sebesség AKKOR
      mozgasd a rovert az útvonal megfelelő pontjára
      frissítsd a megtett cellák számát
      frissítsd az akkut
      naplózd a MOVING műveletet
      lépj a következő körre
    VÉGE
```

```
  VÉGE
VÉGE
HAZAFELÉ:
HA a rover nincs otthon AKKOR
  keress útvonalat haza (astar)
  HA van hazafelé útvonal AKKOR
    válassz biztonságos hazafelé sebességet (biztSebesseg)
    HA létezik megfelelő sebesség AKKOR
      mozgasd a rovert hazafelé
      frissítsd a megtett cellák számát
      frissítsd az akkut
      naplózd a MOVING műveletet
      lépj a következő körre
    VÉGE
  VÉGE
VÉGE
várakozás (STANDING)
naplózd a STANDING műveletet
lépj a következő körre
```

A terv ellenőrzése

A rover intelligenciája nem az A* útkeresés brute force-ra alakul, hanem az, hogy egy tervet előre le tud ellenőrizni. Ez a következő függvényben történik:

tervSzimulacio

A függvény azt vizsgálja meg, hogy egy terv végrehajtható-e:

- eljutás a célhoz
- bányászás
- visszatérés

A terv szimulációja során minden körben több akciójelöltet vizsgál:

- várakozás
- mozgás 1-es sebességgel
- mozgás 2-es sebességgel

- mozgás 3-as sebességgel
- bányászás

Majd kiválasztja a legjobb műveletet.

Terv szimuláció

Eljárás: tervSzimulacio

Bemenet:

- kezdő idő
- kezdő akkumulátor
- hátralévő odafelé lépések
- kell-e még bányászni
- hátralévő visszaút
- küldetés végső időkorlátja

Kimenet:

- végrehajtható-e a terv
- meddig jut a szimuláció
- mennyi akku marad

Pszeudokód:

```
állítsd be az aktuális időt a kezdő időre
állítsd be az aktuális akkut a kezdő akku értékére

AMÍG az aktuális idő kisebb, mint a maximális idő
  HA már nincs odafelé út ÉS nem kell bányászni ÉS nincs visszaút AKKOR
    Térj vissza sikerrel
  VÉGE
  állapítsd meg, hogy nappal van-e (nappal_van)
  számold ki, hogy a következő művelet után mennyi idő marad
  kezdetben nincs kiválasztott művelet, a legjobb maradó akku = -1e100, haladás = -1
```

1. Várakozás vizsgálata

```
HA a várakozás szabályos (allFunc) AKKOR
  számold ki a várakozás utáni akkut (allFunc)
  HA a maradék terv minimális időigénye belefér (teljesUtUtes) AKKOR
    jelöld ezt lehetséges műveletnek
  VÉGE
VÉGE
```

2. Mozgás vizsgálata

```
HA még van odafelé út VAGY már csak a visszaút maradt AKKOR
  CIKLUS sebesség = 1..3
    HA a sebesség nagyobb, mint a megtehető lépések: ugord át
    HA a mozgás nem szabályos (mozoghat): ugord át
    csökkentsd a megfelelő szakasz hátralévő lépésszámát
    számold ki az új akkut (mozogFunc)
    HA a maradék terv minimális ideje belefér (teljesUtUtes) AKKOR
      hasonlítsd össze a jelenlegi legjobbal
      előnyben részesítve: 1. nagyobb maradó akku  2. nagyobb haladás
    VÉGE
  CIKLUS VÉGE
VÉGE
```

3. Bányászás vizsgálata

```
HA már odaértünk ÉS még kell bányászni AKKOR
  HA a bányászás szabályos (banyaszhat) AKKOR
    számold ki a bányászás utáni akkut (banyaszFunc)
    HA a maradék terv minimális ideje belefér (teljesUtUtes) AKKOR
      hasonlítsd össze a jelenlegi legjobbal
    VÉGE
  VÉGE
VÉGE
```

HA nincs egyetlen érvényes művelet sem: térj vissza sikertelenséggel

alkalmazd a kiválasztott legjobb akciót, növeld az időt eggyel

CIKLUS VÉGE

ha lejárt az idő és a terv még nincs kész: térj vissza sikertelenséggel

Biztonságos célpont kiválasztása

Eljárás: celpontValasztas

Bemenet:

- rover állapota
- küldetés állapota
- térkép
- hazatérési távolságok

Kimenet:

- van-e megfelelő cél
- ha igen, annak koordinátái

Pszeudokód:

kezdetben nincs legjobb célpont (-1)

járd be a teljes térképet

HA az adott cella nem ásvány: ugord át

HA az adott cellából a bázis nem elérhető (hazTav == 255): ugord át

keress pontos útvonalat a rover helyétől a célcelláig (astar)

HA nincs útvonal: ugord át

számold ki az odaút és a hazaút hosszát

futtasd le a teljes terv szimulációját (tervSzimulacio)

HA a terv nem végrehajtható: ugord át

hasonlítsd össze a célpontot a jelenlegi legjobbal

előnyben részesítve:

1. rövidebb teljes oda-vissza út
2. azonos esetben rövidebb odaút
3. azonos esetben nagyobb végső akku

HA találtál megfelelő célpontot: térj vissza annak koordinátáival

KÜLÖNBEN: térj vissza sikertelenséggel

Biztonságos sebesség kiválasztása

Megjegyzés: 3. sebességet igyekeztem kerülni, mivel tapasztalatom szerint hosszabb utak során problémát okozott az akkumulátorral. Nappal is ha 3. sebességgel mozgunk akkor mínuszban maradunk, így meghoztam azt a döntést, hogy biztonság és hazatérés legyen az első helyen. Lásd a kivétel esetet.

Eljárás: biztSebesseg

Bemenet:

- rover aktuális állapota
- küldetés állapota
- útvonal
- hátralévő lépések száma
- cél típusa
- cél koordinátái

Kimenet:

- választott sebesség (vagy 0, ha most nem szabad mozdulni)

Pszeudokód:

állapítsd meg, hogy nap van-e (nappal_van)

állíts fel egy alap sebességsorrendet:

ha nappal van: preferencia: 2, 1, 3

különben: preferencia: 1, 2, 3

számold ki a maradék teljes terv minimális időigényét (teljesUtUtes)

ha nagyon szoros az idő: sorrend legyen 3, 2, 1

végigpróbálod a sebességeket a sorrend szerint:

HA sebesség > hátralévő lépések: ugord át

HA a mozgás nem szabályos (mozoghat): ugord át

számold ki a mozgás utáni akkut (mozogFunc)

szimuláld a hátralévő teljes tervet (tervSzimulacio)

HA a terv így még végrehajtható: térj vissza ezzel a sebességgel

ha egyik sebesség sem megfelelő: térj vissza 0-val

Miért nem csak A* az AI?

Fontos hangsúlyozni, hogy az AI algoritmus nem azonos az A* útkereséssel.

Az A* csak azt mondja meg:

- hogyan lehet eljutni két pont között.

Az AI viszont ennél sokkal többet csinál:

- kiválasztja, hogy érdemes-e egyáltalán odamenni
- ellenőrzi, hogy a célpont vállalható-e
- ellenőrzi, hogy a bányászás után is marad-e esély a hazatérésre
- kiválasztja a sebességet
- eldönti, hogy inkább várjon-e
- szükség esetén feladja az ásványgyűjtést és hazafelé indul

előrelátó tervellenőrzés + biztonságos döntéshozatal

Az AI algoritmus lényege egy mondatban

A rover AI minden körben olyan műveletet választ, amely nemcsak az adott pillanatban szabályos, hanem előretekintve is biztosítja, hogy a teljes terv időben, elegendő energiával és biztonságos hazatéréssel végrehajtható maradjon

Rövid összefoglalás

A rover AI algoritmusa négy fő szintből áll:

Szint	Leírás	Függvény(ek)
1. Útvonalinformáció előállítása	BFS + A* útkereső	hazaTavBFS astar
2. Idő- és energiaigény becslése	Minimális ütések számítása	teljesUtUtes

3. Jövőbeli terv megvalósíthatóságának ellenőrzése	Előretekintő szimuláció	tervSzimulacio
4. Konkrét döntés és végrehajtás	Célpont + sebesség + lépés	celpontValasztas biztSebesseg szimulacioLepes